

Advanced Swift

Module 4

GSS

Reading & Writing JSON

- Coding and decoding data is a critical part of many applications, allowing them to store and retrieve information as needed.
- coding and decoding data is made through the use of the `Codable` protocol.
- The `Codable` protocol combines the `Encodable` and `Decodable` protocols.
- In Swift, the most commonly used format for coding and decoding data is JSON, which is a lightweight, human-readable data interchange format.
- The standard library provides the `JSONEncoder` and `JSONDecoder` classes for encoding and decoding JSON data, respectively.

Reading & Writing JSON

- With `Codable` and `JSONSerialization`, Swift provides an efficient and convenient way to handle the coding and decoding of data in your applications.
- Whether you're storing user information, saving game data, or working with APIs, coding and decoding data is a crucial part of many applications and is made simple with the use of `Codable` in Swift.

Codable Protocol

- The `Codable` protocol in Swift is a combination/union of the `Encodable` and `Decodable` protocols that allow for encoding and decoding of custom data types to and from external representation, such as JSON.
- By conforming to `Codable`, a custom data type can be easily serialized and deserialized for storage or network transmission, without having to manually write encoding and decoding logic.
- The Swift standard library provides built-in support for `Codable` by automatically handling the encoding and decoding of common data types, such as strings, numbers, and arrays.
- The `JSONEncoder` and `JSONDecoder` classes are used to encode and decode `Codable` data to and from JSON format, respectively.

Codable Protocol

- The syntax for conforming to the Codable protocol in Swift is as follows:

```
import Foundation
struct Car: Codable {
    let make: String
    let model: String
    let year: Int
    let color: String
}
```

Or , if conforming to Codable with custom keys:

```
struct Car: Codable {
    let make: String
    let model: String
    let year: Int
    let color: String

    enum CodingKeys: String, CodingKey {
        case make = "manufacturer"
        case model = "vehicle_model"
        case year
        case color
    }
}
```

Serializing to JSON

- JSON (JavaScript Object Notation) is a lightweight data interchange format that is easy for humans to read and write, and easy for machines to parse and generate.
- It is a text-based, structured data format that uses conventions from the JavaScript programming language, and is commonly used for asynchronous communication between clients and servers, as well as for exchanging data between applications.
- JSON is widely used because of its simplicity, readability, and support in many programming languages, including Swift.

A JSON object might look like this:

```
{  
  "name": "John Doe",  
  "age": 30,  
  "email": "johndoe@example.com",  
  "address":  
    { "street": "123 Main St.",  
      "city": "San Francisco",  
      "state": "CA",  
      "zip": "94109"  
    }  
}
```


Serializing to JSON

- JSON (JavaScript Object Notation) is a lightweight data interchange format that is easy for humans to read and write, and easy for machines to parse and generate.
- It is a text-based, structured data format that uses conventions from the JavaScript programming language, and is commonly used for asynchronous communication between clients and servers, as well as for exchanging data between applications.
- JSON is widely used because of its simplicity, readability, and support in many programming languages, including Swift.

A JSON array might look like this:

```
[
  {
    "name": "John Doe",
    "age": 30,
    "email": "johndoe@example.com"
  },
  {
    "name": "Jane Doe",
    "age": 28,
    "email": "janedoe@example.com"
  }
]
```


Serializing to and from JSON in swift

- Here's are examples of **encoding** and **decoding** Car using the **JSONEncoder** and **JSONDecoder** classes in Swift:

```
// Encoding to JSON
let car = Car(make: "Toyota", model: "Camry", year: 2020, color: "Silver")
let encoder = JSONEncoder()
let data = try encoder.encode(car)
let jsonString = String(data: data, encoding: .utf8)
print(jsonString)

// Output: Optional("{\"manufacturer\":\"Toyota\",\"vehicle_model\":\"Camry\",\"year\":2020,\"color\":\"Silver\"}")
```

- In this example, the **Car** struct conforms to the **Codable** protocol, which makes it easy to encode into JSON.
- An instance of **JSONEncoder** is created and the **encode** method is called with the **Car** instance as an argument.
- This will return a **Data** object that contains the JSON representation of the **Car** instance.

Serializing to and from JSON in swift

- To **decode** a basic data type, you create an instance of the `JSONDecoder` class and call its `decode` method with the type that you want to decode as the argument. For example:

```
// Decoding from JSON

let decoder = JSONDecoder()
let decodedCar = try decoder.decode(Car.self, from: data)
print(decodedCar)

// Output: Car(make: "Toyota", model: "Camry", year: 2020, color: "Silver")
```

- In this example, an instance of `JSONDecoder` is created and its `decode` method is called with `Car.self` as the argument and the `data` object as the source of the JSON data.
- This will return an instance of the `Car` struct that has been initialized with the values contained in the JSON data.

Decoding Arrays in Swift

- When decoding arrays in Swift, you are converting a JSON-style array data structure into Swift objects.
- To decode an array, you use the decode method of the JSONDecoder class, passing in the type of the array as a generic parameter.
- For example, Consider this JSON data:

```
[
  {
    "make": "Tesla",
    "model": "Model S",
    "year": 2020,
    "color": "Red"
  },
  {
    "make": "Toyota",
    "model": "Camry",
    "year": 2019,
    "color": "Silver"
  }
]
```

Decoding Arrays in Swift

Consider this JSON data:

```
[
  {
    "make": "Tesla",
    "model": "Model S",
    "year": 2020,
    "color": "Red"
  },
  {
    "make": "Toyota",
    "model": "Camry",
    "year": 2019,
    "color": "Silver"
  }
]
```

Here's the struct that you'll use to decode the data:

```
import Foundation

struct Car: Codable {
    let make: String
    let model: String
    let year: Int
    let color: String
}
```

```
[ let jsonData = """
[
  {
    "make": "Tesla",
    "model": "Model S",
    "year": 2020,
    "color": "Red"
  },
  {
    "make": "Toyota",
    "model": "Camry",
    "year": 2019,
    "color": "Silver"
  }
]
"""
jsonData.data(using: .utf8)!
```

```
let decoder = JSONDecoder()
let cars = try decoder.decode([Car].self, from: jsonData)
print(cars)
// Output: [Car(make: "Tesla", model: "Model S", year: 2020, color: "Red"),
// Car(make: "Toyota", model: "Camry", year: 2019, color: "Silver")]
```


Handle Custom Keys & Missing Values while Decoding in Swift

- Handling custom keys and missing values during decoding in Swift involves using `CodingKeys` and the `decodeIfPresent` method, respectively.
- The `CodingKeys` enumeration is used to map between the property names in the Swift code and the keys in the JSON data.
- For example, if the key in the JSON data is different from the property name in the Swift code, the property name can be mapped to the correct JSON key using the `CodingKeys` enumeration.

```
{  
  "make_name": "Toyota",  
  "model_name": "Camry",  
  "year_made": 2019,  
  "color": "Black"  
}
```

Handle Custom Keys & Missing Values while Decoding in Swift

- Here, the keys in the JSON data are different from the property names in the Swift code. To map these keys, the following CodingKeys enumeration can be used:

```
import Foundation
struct Car: Codable {
    let make: String
    let model: String
    let year: Int
    let color: String

    private enum CodingKeys: String, CodingKey {
        case make = "make_name"
        case model = "model_name"
        case year = "year_made"
        case color
    }
}
```

Handle Custom Keys & Missing Values while Decoding in Swift

- You can now decode the JSON as follows:

```
let jsonData = """
[
  {
    "make_name": "Toyota",
    "model_name": "Camry",
    "year_made": 2019,
    "color": "Black"
  }
]
""".data(using: .utf8)!

let decoder = JSONDecoder()
let cars = try decoder.decode([Car].self,
from: jsonData)
print(cars)

// Output: [Car(make: "Toyota", model:
"Camry", year: 2019, color: Optional("Black"))]
```


Handling Missing Values during Decoding

- Regarding handling missing values during decoding, Swift provides the **decodeIfPresent** method that allows to decode an optional value and return nil if the value is not present in the JSON data. For example:

```
struct Car: Codable {
    let make: String
    let model: String
    let year: Int
    let color: String?

    init(from decoder: Decoder) throws {
        let container = try decoder.container(keyedBy: CodingKeys.self)
        make = try container.decode(String.self, forKey: .make)
        model = try container.decode(String.self, forKey: .model)
        year = try container.decode(Int.self, forKey: .year)
        color = try container.decodeIfPresent(String.self, forKey: .color)
    }
}
```

- In the above example, the color property is declared as an optional and its value is decoded using decodeIfPresent().
- If the color is not present in the JSON data, the value of color will be nil.

Handling Missing Values during Decoding

- Let's see what happens if you try to decode some JSON that does not contain the `color` property:

```
let jsonData = """
[
  {
    "make": "Toyota",
    "model": "Camry",
    "year": 2019,
  }
] """
.data(using: .utf8)!
let decoder = JSONDecoder()
let cars = try decoder.decode([Car].self, from: jsonData)
print(cars)
// Output: [Car(make: "Toyota", model: "Camry", year: 2019, color:
nil)]
```

- The JSON successfully decodes, but `color` is set to `nil`, as expected.